

试错学习：面向 LLM 智能体的 基于探索的轨迹优化 (ETO)

Yifan Song, Da Yin, Xiang Yue, Jie Huang, Sujian Li, Bill Yuchen Lin
Allen Institute for AI 北京大学 (Peking University) UCLA Ohio State University UIUC

英文原标题： Trial and Error: Exploration-Based Trajectory Optimization for LLM Agents
发表： ACL 2024 主会 (Main Conference) **引用数：** 约 181 (Semantic Scholar, 2026-06-07)
代码与数据： <https://github.com/Yifan-Song793/ETO>
一句话定位： 让开源大模型智能体从自己的失败里学习——先模仿专家学会基本操作，再到环境里真刀真枪地试、收集失败案例，把“失败轨迹”和“成功轨迹”配成对，用 DPO 这种对比学习的方式“奖善惩失”，并可反复迭代越练越强。

小白知识卡：读这篇前先搞懂这几个词

智能体 (agent) 这里指一个能“看环境—想一步—做动作—再看反馈”循环操作的大模型程序。比如让它去网购、做虚拟科学实验、在虚拟房间里做家务，它要一步步执行直到完成任务。

轨迹 (trajectory) 智能体完成一个任务从头到尾的完整记录：任务指令 + 第 1 个动作 + 环境反馈 + 第 2 个动作 + ……一直到结束。一条轨迹就是“这一局是怎么打的”的全过程。

奖励 (reward) 环境给这条轨迹打的分，一般是 0 到 1。1 表示任务完美完成，0 表示失败。它是判断“成功还是失败”的唯一依据。

模仿学习 / 行为克隆 (imitation learning / behavioral cloning, BC) 最朴素的训练法：给模型看一堆“专家是怎么做的”示范，让它照着学、把专家的动作背下来。本文里就是对专家轨迹做**监督微调 (supervised fine-tuning, SFT)**。缺点：只会模仿，没真正去环境里探索过，遇到没见过的情況容易翻车。

ReAct 与思维链 (Chain-of-Thought, CoT) ReAct 是一种让智能体“边想边做”的格式：每做一个动作前，先用自然语言写一段推理（也就是 CoT），再给出动作。本文所有智能体都用这种“先想后做”的格式。

强化学习 (reinforcement learning, RL) 与 PPO 让智能体自己在环境里试、根据拿到的奖励来调整策略的训练范式。PPO (Proximal Policy Optimization, 近端策略优化) 是最常用的在线 RL 算法。它能真正“探索”，但在大模型上**训练不稳定、效率低**，很难调。

DPO (Direct Preference Optimization, 直接偏好优化) 一种“绕开 RL 复杂流程”的训练法。你只要给它一对数据“A 比 B 好”，它就能直接调模型：**抬高好样本的概率、压低坏样本的概率**。本文的核心武器：把“成功轨迹 vs 失败轨迹”当成这样一对偏好数据喂给 DPO。

信用分配 (credit assignment) 一条轨迹好几十步，最后只给一个总分——到底是哪一步做对/做错导致了这个结果？把功劳或锅分到具体某一步，就叫信用分配。这是序列决策里出了名的难题，本文末尾也坦承自己在这点上做了简化。

分布外泛化 (out-of-distribution, OOD / unseen) 测试时遇到训练时没见过的新任务变体。比如训练时学的是“烧开水”，测试时却要“熔化铅”。能不能在这种新场景上也做好，衡量的就是泛化能力。

1 摘要全译

大语言模型 (Large Language Models, LLMs) 已经成为各类自主智能体 (autonomous agent) 系统不可或缺的组成部分。在本研究中，我们提出一种基于探索的轨迹优化方法，称为 ETO (Exploration-based Trajectory Optimization)。这一学习方法旨在提升开源大模型智能体的性能。与以往**只在成功的专家轨迹上训练**的研究不同，我们的方法让智能体能够从**自身探索的失败**中学习。借助一个迭代式的优化框架，这带来了性能上的提升。在探索阶段，智能体一边完成给定任务、一边与环境交互，收集失败轨迹，从而构造出成对的“对比轨迹”。在随后的训练阶段，智能体利用这些轨迹偏好对，采用 DPO (Rafailov et al., 2023) 等对比学习方法来更新自己的策略。这种“探索—训练”的迭代循环促使智能体持续进步。我们在三项复杂任务上的实验表明，ETO 始终大幅超越基线方法的表现。此外，对“任务求解效率”以及“缺少专家轨迹场景下的潜力”的考察，进一步印证了本方法的有效性。

2 引言：为什么要让智能体“吃一堑长一智”

背景痛点。大模型在网页浏览、具身任务 (embodied tasks, 指在模拟的物理/家居环境里操作)、多模态推理、复杂问答等交互任务上展现了惊人能力。但用 ChatGPT、GPT-4 当“大脑”毕竟成本高、不可控；而**开源大模型当智能体时，效果比 GPT-4 差很多** (Liu et al., 2023 等)。所以业界希望把开源模型训练成好用的智能体。

现有做法的局限。主流做法是**模仿学习**：拿专家轨迹对开源模型做 SFT (如 FireAct、AgentTuning、Lumos 等)。这相当于让模型“背标准答案”。问题在于：它**完全依赖专家示范，从没真正去目标环境里探索过**，于是学到的往往是次优策略，泛化能力也受限——遇到稍有变化的新任务就容易出错。

本文的灵感与思路。人类学习不仅靠看成功示范，更靠在**试错 (trial and error) 中亲历失败**。受此启发，作者提出 ETO：不再只用成功轨迹，而是**利用当前策略探索时产生的失败**来强化学习过程。整套流程是 (见图 1)：

1. 先用 SFT 行为克隆训练出一个**基础智能体** (base agent)；

2. **探索阶段**：让这个基础智能体去目标环境里实际跑一遍训练任务，收集它**失败的轨迹**，并和之前已有的**专家成功轨迹**配成对；
3. **训练阶段**：用 DPO 损失 (Rafailov et al., 2023)，拿这些“失败 vs 成功”的对比轨迹对去微调模型，让它**偏好成功、厌恶失败**；
4. 上述过程可**多轮迭代**：用上一轮练出来的智能体再去探索、再收集新失败、再训练，循环往复。

实验与结论速览。作者在三个代表性数据集上验证：WebShop (网页购物导航)、ScienceWorld (模拟科学实验)、ALFWorld (具身家务)。结果是：ETO 始终大幅超过 SFT 行为克隆和其他强基线；在 ScienceWorld 的分布外 (OOD) 测试集上比 SFT 提升达 22%，泛化能力突出；它还能用更少的动作步数拿到更高奖励 (效率更高)；甚至在**完全没有专家轨迹**的极端情形下，靠“自我对弈 (self-play)”也能有不错表现。

三点贡献。(1) **方法**：提出 ETO，一种迭代收集失败轨迹、用对比学习精炼策略的算法；(2) **评测**：在三项复杂交互任务上全面实验，超越 SFT 与其他强基线；(3) **分析**：从分布外泛化、动作效率、以及“无需专家轨迹的可行性”等多个角度深入验证有效性。

3 任务形式化：把“做任务”写成数学语言

作者把“带环境反馈的智能体任务”形式化为一个**部分可观测马尔可夫决策过程** (partially observable Markov decision process, POMDP)，记作六元组 $(\mathcal{U}, \mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{T}, \mathcal{R})$ ：

- $\mathcal{U}$ ：指令空间 (任务怎么描述)； $\mathcal{S}$ ：状态空间 (环境真实状态)； $\mathcal{A}$ ：动作空间； $\mathcal{O}$ ：观测空间 (智能体能看到的反馈)；
- $\mathcal{T} : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ ：转移函数 (做了动作后状态怎么变)； $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ ：奖励函数。
- **关键点**：在大模型智能体场景里， $\mathcal{U}, \mathcal{A}, \mathcal{O}$ 都是**自然语言**的子集——也就是说，指令、动作、观测都是文字。

给定一条任务指令 $u \in \mathcal{U}$ ，参数为 θ 的大模型智能体按其策略 π_θ 生成第一个动作 $a_1 \sim \pi_\theta(\cdot|u)$ 。这个动作会改变隐状态 s_t 、并返回一个观测 o_t ；接着在第 $t+1$ 步生成 $a_{t+1} \sim \pi_\theta(\cdot|u, a_1, o_1, \dots, o_{t-1}, a_t)$ 。如此循环，直到任务完成或超过最大步数。整条轨迹记为：

$$e = (u, a_1, o_1, \dots, o_{n-1}, a_n) \sim \pi_\theta(e|u), \quad (1)$$

$$\pi_\theta(e|u) = \prod_{j=1}^n \pi_\theta(a_j|u, a_1, o_1, \dots, o_{j-1}), \quad (2)$$

其中 n 是轨迹长度。**公式 (2) 拆解 (很重要)**：一整条轨迹的概率，就等于**逐步动作概率连乘**——第 j 步动作 a_j 是在“看到此前所有动作和观测”的条件下生成的。注意连乘里只算**动作** a_j 的概率，

不算环境给的观测 o （因为观测不是模型生成的）。最后环境算出总奖励 $r(u, e) \in [0, 1]$ ，等于 1 即任务成功。

4 方法：ETO 怎么从失败里学习

ETO 先用行为克隆造一个基础智能体，然后在此基础上迭代式地“从试错中精炼策略”。

4.1 第一步：行为克隆（打底）

行为克隆（BC）就是在专家交互轨迹上做 SFT，它是搭建强智能体的可靠起点。本文用 ReAct 风格的轨迹来做 BC——即每个动作前都额外生成一段 CoT 推理。为简化记号，下文用 a 同时代表“CoT + 动作”这一整块。

给定专家轨迹数据集 $D = \{(u, e)^{(i)}\}_{i=1}^{|D|}$ ，在自回归损失上微调出基础智能体 π_{base} ：

$$\mathcal{L}_{\text{SFT}}(\pi_{\theta}) = -\mathbb{E}_{e \sim D} [\pi_{\theta}(e|u)], \quad (3)$$

其中 $e = (u, a_1, o_1, \dots, o_{n-1}, a_n) \sim D$ 是一条专家轨迹。具体实现时，把指令、动作、观测拼成一段文本序列 $t = \text{concat}(u, a_1, o_1, \dots, o_{n-1}, a_n) = (t_1, t_2, \dots, t_l)$ ， t_k 是第 k 个 token。于是轨迹概率可由下式得到（把任务描述和观测部分的 token 都“遮住”不算损失）：

$$\pi_{\theta}(e|u) = -\sum_k \log \pi_{\theta}(t_k | t_{<k}) \times \mathbb{I}(t_k \in \mathcal{A}), \quad (5)$$

公式 (5) 拆解： $\mathbb{I}(t_k \in \mathcal{A})$ 是一个指示函数——只有当这个 token 属于智能体自己产出的动作时才取 1，否则取 0。换句话说，训练时只对“模型该说的话（动作/CoT）”计算损失，对指令和环境观测不计损失。这很合理：环境反馈不是模型该背的，强行学反而有害。

4.2 第二步：从探索失败中学习（核心）

为什么不直接上 RL？ 行为克隆只会模仿专家、不会探索，策略往往次优。要练得更强，理应让模型也去探索失败轨迹——最直接的办法就是强化学习，其目标是：

$$\max_{\pi_{\theta}} \mathbb{E}_{u \sim D, e \sim \pi_{\theta}(e|u)} [r(u, e)] - \beta D_{\text{KL}} [\pi_{\theta}(e|u) \parallel \pi_{\text{ref}}(e|u)], \quad (6)$$

公式 (6) 拆解：前一项是最大化期望奖励（多拿高分），后一项是一个带权重 β 的 KL 散度约束，作用是别让新策略 π_{θ} 偏离参考策略 π_{ref} （也就是基础智能体 π_{base} ）太远，防止训崩。实践中待训练的 π_{θ} 也用 π_{base} 初始化。这个目标可以用 PPO 等 RL 方法来解。

但是——在大模型智能体上直接做在线 RL 有现实困难：不稳定、效率低。因此作者改走迭代式离线学习路线：用对比轨迹数据来训练，把整个过程组织成“探索—训练”循环（见图 1）。

探索阶段 (Exploration Phase)。 基础智能体 π_{base} 在 BC 用过的那批训练指令上去环境里跑一遍，得到自己的轨迹：

$$\hat{e} = (u, \hat{a}_1, \hat{o}_1, \dots, \hat{o}_{m-1}, \hat{a}_m) \sim \pi_{\text{base}}(e|u). \quad (7)$$

环境为这条轨迹 $\hat{e}$ 返回奖励 $\hat{r}$ 。然后构造“失败—成功”轨迹对，记作 $e_w \succ e_l | u$ ： e_w 是高奖励（胜出）轨迹、 e_l 是低奖励（落败）轨迹，分别从**专家轨迹** e 和**智能体自己跑出的轨迹** $\hat{e}$ 中按奖励高低选出。**注意两条规则**：（1）只收集**两条轨迹奖励不同的对**；（2）如果智能体和专家**都成功了**，这一对就直接**丢弃**（没有对比价值）。最终得到对比轨迹数据集 $D_p = \{(u, e_w, e_l)^{(i)}\}_{i=1}^{|D_p|}$ 。

训练阶段 (Training Phase)。 用轨迹对里的“成败对比信息”来更新策略。给定 $e_w \succ e_l | u$ ，这种成败关系用 **Bradley-Terry (BT) 模型**建模：

$$p(e_w \succ e_l | u) = \frac{\exp(r(u, e_w))}{\exp(r(u, e_w)) + \exp(r(u, e_l))}. \quad (8)$$

公式 (8) 直觉：奖励越高的轨迹，“它更优”的概率就越大——这就是个用奖励差异决定胜负概率的 softmax。再借助 DPO 的关键推导：在目标 (6) 的最优策略 π_r 下，奖励可写成 $r(u, e) = \beta \log \frac{\pi_r(e|u)}{\pi_{\text{ref}}(e|u)} + \beta \log Z(u)$ （其中 $Z(u)$ 是配分函数）。把它代回 (8)，配分函数恰好抵消，得到**直接用策略表示的 BT 模型**，从而推出 DPO 损失：

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(u, e_w, e_l) \sim D_p} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(e_w|u)}{\pi_\theta(e_l|u)} - \beta \log \frac{\pi_{\text{ref}}(e_w|u)}{\pi_{\text{ref}}(e_l|u)} \right) \right]. \quad (11)$$

公式 (11) 拆解 (全文最关键的一条)： σ 是 sigmoid 函数。括号里在比较两件事——“当前策略 π_θ 对成功轨迹 vs 失败轨迹的相对偏好”，减去“参考策略 π_{ref} 的同样偏好”。优化它的效果就是：**抬高成功轨迹 e_w 的概率、压低失败轨迹 e_l 的概率**，同时用参考策略那一项当“锚”，约束模型别跑偏。妙处在于：**这其实是 RL 目标 (6) 的等价改写**，它在直接最大化最终奖励，却**完全不用做 RL 那套不稳定的优化**。

4.3 第三步：迭代（越练越强）

训练完一轮后，更新过的策略可以**再去收集新的失败案例**、构造新的对比对，再训练一轮。完整流程见算法 1 (Algorithm 1)：先做 T_1 步行为克隆得到基础智能体；然后做 I 轮迭代，每轮里令 $\pi_{\text{base}} = \pi_{\text{ref}} = \pi_\theta$ ，跑出智能体轨迹、与专家轨迹比奖励配成对、构造 D_p ，再做 T_2 步 DPO 训练。如此循环输出最终策略 π_θ 。

5 方法图解：一张图看懂数据怎么流动

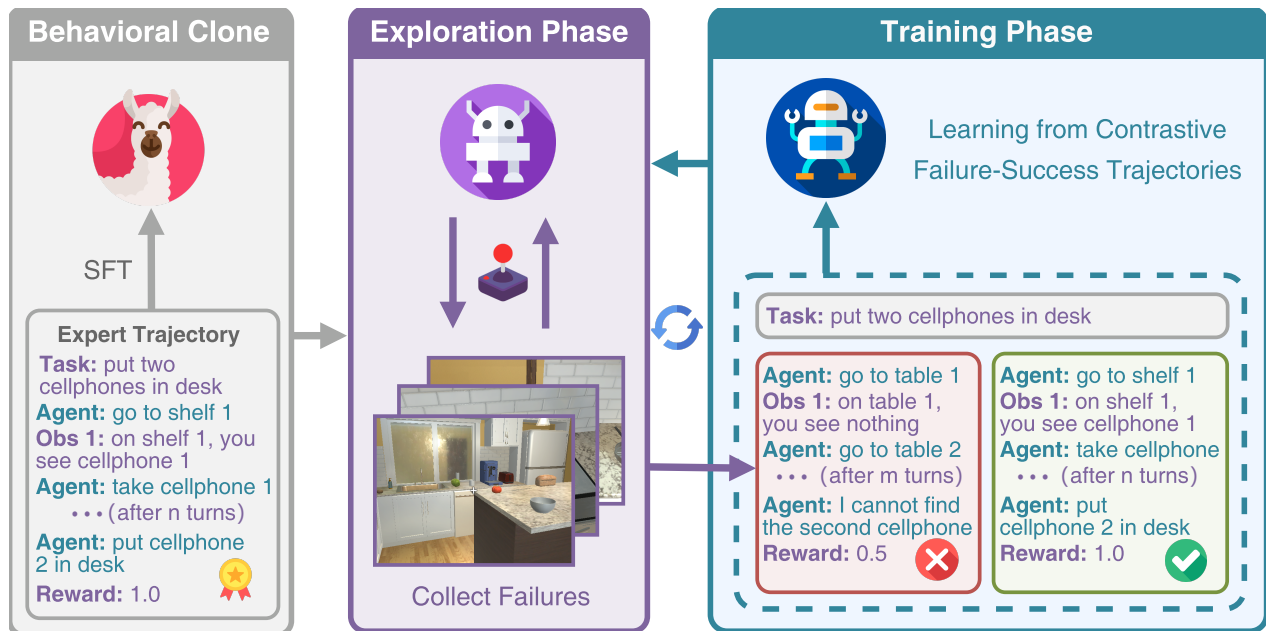


图 1: ETO 总览 (原文图 2)。从一个经行为克隆训练得到的基础大模型智能体出发, 本方法让智能体迭代地收集失败轨迹, 并通过不断从“失败—成功”对比轨迹对中学习来更新策略。

图里在说什么、数据怎么流动? 整张图从左到右是三个阶段, 用一个具体任务“把两部手机放到桌上 (put two cellphones in desk)”串起来:

- **左: 行为克隆 (Behavioral Clone)**。拿一条专家成功轨迹 (Expert Trajectory, 最后 Reward=1.0, 带金牌图标) 对模型做 SFT, 得到一个会基本操作的基础智能体 (图中是只羊驼/Llama 的形象)。
- **中: 探索阶段 (Exploration Phase)**。让这个基础智能体 (紫色机器人) 真的去环境里跑这些任务, 把它失败的轨迹收集起来 (Collect Failures)。图中下方一摞屏幕就代表它跑出的各种 (多半失败的) 尝试。
- **右: 训练阶段 (Training Phase)**。把失败轨迹 (左列, Reward=0.5, 红叉) 和成功轨迹 (右列, Reward=1.0, 绿勾) 并排配成一对, 喂给 DPO 做对比学习 (“Learning from Contrastive Failure-Success Trajectories”)。对照两列能直观看出差异: 失败的那次先去了 table 1/table 2 都没找到第二部手机, 最后只拿 0.5 分; 成功的那次去了 shelf 1 拿到手机、顺利放进桌子, 拿满 1.0 分。模型就是从这种“同一个任务、一成一败”的对照里学到该怎么做、不该怎么做。
- **回路箭头 (中间的循环图标)**: 训练完的新策略可以再回到探索阶段收集新失败, 形成迭代闭环。

一句话总结这张图: 先模仿打底 → 放出去试错收集失败 → 把成败配对用 DPO 对比学习 → 循环加强。

6 实验解读：数字说明了什么

6.1 用了什么数据、比了哪些基线

三个交互环境（数据集见表 1）：

- **WebShop**：网购环境，按指令在购物网站上搜索、点选、下单，奖励按商品属性和价格的匹配度给（0~1 连续值）。
- **ScienceWorld**：文字版虚拟环境，做小学级科学实验（热力学、电路等 10 类任务），每个任务有若干子目标，奖励按完成的子目标算。**测试集是“关键未见变体”**（如训练学“烧开水”、测试要“熔化铅”），专门考泛化。
- **ALFWorld**：具身家务环境，完成“把某物放到某处”这类指令；**只给二元奖励**（成功 =1/失败 =0），没有中间分。它分 seen（同分布）和 unseen（分布外）两个测试集。

表 1：数据集统计（原文表 1）。“Test-Seen / Test-Unseen”分别是“见过场景 / 未见场景”的测试集；“#Turns”是专家轨迹的平均交互轮数。

数据集	#Train	#Test-Seen	#Test-Unseen	#Turns
WebShop	1938	200	—	4.9
ScienceWorld	1483	194	241	14.4
ALFWorld	3321	140	134	10.1

对比的基线方法：（基座主要用 Llama-2-7B-Chat）

- **SFT**：纯行为克隆，也是 ETO 和其他方法的“基础智能体”。
- **Best-of-N**：用 SFT 智能体采样 $N = 10$ 条，挑奖励最高那条。
- **RFT (Rejection sampling Fine-Tuning, 拒绝采样微调)**：一个强基线——把**成功的**采样轨迹加进专家数据集再训练。注意它**只学成功轨迹**。
- **PPO**：直接用 RL 最大化最终奖励的在线方法。
- 还对比了 GPT-3.5-Turbo、GPT-4、以及没微调过的 Llama-2-7B-Chat。

评测设置：统一用 ReAct 格式（先 CoT 再动作），加 1-shot 示例，解码温度设 0.0（确定性输出，Best-of-N 除外）。主指标是**平均奖励 (Average Reward)**。训练用 8 张 A100-80G；ETO 迭代轮数 WebShop/ScienceWorld 为 2、ALFWorld 为 1；DPO 的 β 在 WebShop/ScienceWorld 取 0.1、ALFWorld 取 0.5。

6.2 主结果：ETO 全面领先

表 2: 各方法在三个数据集上的**平均奖励** (原文表 2)。“Seen” 是任务类型在训练中见过的留出测试集; “Unseen” 是含关键未见任务变体的测试集。最优结果加粗。

方法	WebShop	ScienceWorld		ALFWorld	
		Seen	Unseen	Seen	Unseen
GPT-4	63.2	64.8	64.4	42.9	38.1
GPT-3.5-Turbo	62.4	16.5	13.0	7.9	10.5
Llama-2-7B-Chat	17.9	3.8	3.1	0.0	0.0
Llama-2-7B-Chat + SFT	63.1	67.4	53.0	60.0	67.2
Llama-2-7B-Chat + Best-of-N	63.8	70.2	57.6	62.1	69.4
Llama-2-7B-Chat + RFT	63.6	71.6	54.3	62.9	66.4
Llama-2-7B-Chat + PPO	64.2	59.4	51.7	22.1	29.1
Llama-2-7B-Chat + ETO (ours)	67.4	73.8	65.0	68.6	72.4

这些数字说明了什么？

- **相比 SFT 大幅提升:** WebShop 平均奖励涨约 8%、ScienceWorld 涨约 9.5%; 在所有数据集上**超过全部基线**。在 WebShop 上 ETO (67.4) **甚至超过 GPT-4** (63.2) ——一个 7B 开源模型在该任务上压过 GPT-4。
- **泛化能力最突出:** 在 ScienceWorld-Unseen 上比 SF (53.0→65.0) 相对提升约 20%。说明“从失败中学”确实能提升对未见场景的适应力。
- **“只学成功”不够:** RFT 虽比 SFT 好, 但因为它只是“成功轨迹的增强版”、**从不学失败**, 提升有限。这反证了**“失败 vs 成功”的对比信息才是关键**。
- **在线 RL (PPO) 不靠谱:** PPO 在 WebShop 略有提升, 但在另两个环境上很差, 尤其 ALFWorld (只有 22.1/29.1) ——因为 ALFWorld 只有**二元奖励**, 加上 RL 本身不稳定, PPO 很难学好。这正是 ETO 用离线 DPO 取代在线 RL 的理由。

6.3 换个基座模型也成立

表 3: 不同基座大模型上的平均奖励（原文表 3）。

基座模型	方法	WebShop	SciWorld	SciWorld
			Seen	Unseen
Llama-2-13B	SFT	66.3	68.1	57.6
	ETO	70.7	71.4	68.6
Mistral-7B	SFT	60.1	63.8	52.2
	ETO	66.2	68.5	62.5

换成 Llama-2-13B 和 Mistral-7B，ETO 依然稳定优于 SFT。两个有趣发现：（1）13B 的相对增益比 7B 小，暗示 ETO 对越弱的智能体帮助越大；（2）Mistral-7B 虽是比较 Llama-2-13B 更强的基础语言模型，但当智能体后却不如 Llama-2-7B——说明“基础语言能力强”和“当智能体强”之间没有强相关。

6.4 效率、迭代次数、对比数据粒度

效率 (图 3)：在 ScienceWorld 里看“多少步拿到多少分”，ETO 能比 SFT 用更少动作步达到更高奖励；个别任务（如 15-90、19-23）甚至比 oracle（标准答案智能体）更早拿到满分 100。结论：从失败中学不仅准，还更高效。

迭代次数 (图 4)：WebShop 和 ScienceWorld 上，前两轮迭代带来提升，但第三轮起开始下降；ALFWorld 只有第一轮有用，二三轮甚至跌破 SFT。原因：专家轨迹集是固定的、探索又在同一批训练集上做，导致对比数据的多样性和数量受限，多轮后会过拟合；ALFWorld 的粗粒度二元奖励更是雪上加霜。作者建议：未来可用 GPT-4 动态构造更多样的对比数据。

对比数据的粒度 (表 4)：作者比较了三种构造对比数据的方式。

表 4: 在 WebShop 上、用不同粒度对比数据训练的智能体平均奖励（原文表 4）。

方法	粒度	lr	β	平均奖励
SFT	—	—	—	63.1
ETO	轨迹级 (Trajectory)	1e-6	0.1	67.4
	步级 (Step)	1e-6	0.1	8.3
	步级 (Step)	1e-7	0.5	62.8
	混合 (Mixture)	1e-6	0.1	64.3

解读：轨迹级对比（整条对整条）效果最好（67.4）。步级对比（比较单步“好动作 vs 坏动作”，见

附录 C 公式 13) 很**不稳定**——需要更小学习率、更大 β 才不崩，且仍只有 62.8。原因是作者只用**最终奖励**去判断某一步动作的好坏，这种对“单步质量”的估计**不准确**（这正是前面知识卡里说的“信用分配”难题）。混合策略也不如纯轨迹级。

6.5 没有专家轨迹怎么办：自我对弈

表 5：无行为克隆（不依赖专家轨迹）的自我对弈性能（原文表 5）。上半部分基于 BC 基础智能体；下半部分直接用未微调的 Llama 当起点。† 表示直接提示未微调的 Llama-2-7B-Chat。

方法	用了 BC?	平均奖励
SFT	是	63.1
RFT	是	63.6
ETO	是	67.4
Llama-2-7B-Chat†	否	17.9
RFT	否	48.4
ETO	否	12.5
RFT+ETO	否	51.2

在**完全没有专家轨迹**的极端设定下（去掉 BC 阶段、让模型用温度 1.0 自己探索、按奖励把自己的不同轨迹配成偏好对）：单用 ETO **反而不行**（12.5）——因为没有 BC 打底，模型连基本操作都不会，自己跑出来的轨迹质量太差。而 RFT 能把 17.9 拉到 48.4。**关键发现：先用 RFT 打底、再上 ETO（RFT+ETO=51.2）效果最好。**启示：没有专家数据时，应先想办法让智能体“能跑出一些成功案例”（如 RFT），再让它从失败中对比学习。

6.6 ETO 与各类方法的定位（表 6）

作者用一张表总结 ETO 相对于其他范式的优势：

表 6：ETO 与各类替代方案的对比（原文表 6）。✓ 表示具备该特性、× 表示不具备。

方法	探索失败	奖励驱动	效率	鲁棒性
SFT	×	×	✓	✓
RFT	✓	×	✓	✓
在线 RL (Online RL)	✓	✓	×	×
离线 RL (Offline RL)	×	✓	✓	×
ETO	✓	✓	✓	✓

只有 ETO 把四样占全了：**能利用探索失败、由奖励驱动、计算高效、且训练鲁棒。**这也是它的

核心卖点——既有 RL 的“探索 + 奖励驱动”，又有离线方法的“高效 + 稳定”。

7 相关工作（简述）

模仿学习：智能体通过模仿专家示范学策略，BC 是其代表；为缓解 BC 的局限有 DAgger (Ross et al., 2011) 等。ETO 与 DAgger **精神相近**（都从失败案例学），但不同点是：DAgger 会在智能体失败处再去收集额外专家轨迹，而 ETO 是从“**成败对比对**”里学，不需要为失败处补标专家数据。**LLM 智能体**：从 AutoGPT、BabyAGI 到 FireAct、AgentTuning、Lumos 等用 GPT-4 造专家数据再 BC 开源模型。**LLM 策略学习**：从偏好中学（如 RLHF）能提升模型；RLHF 用 PPO 但效率低、不稳，DPO 把目标改写成直接建模偏好；ReST、Math-Shepherd 等也探索了迭代/步级奖励建模。

8 结论、局限与对“AI 应用开发”的启示

8.1 核心 takeaways

1. **“从失败中学”是真有用的**：只学成功（SFT/RFT）会留下性能上限；把**失败轨迹与成功轨迹配成对**做对比学习，能显著提升智能体表现和泛化能力（OOD 上提升尤其明显）。
2. **用 DPO 代替在线 RL 是聪明的工程取舍**：DPO 把 RL 的奖励最大化目标改写成“**成败偏好对**”的对比损失，**避开了 PPO 的不稳定与低效**，却拿到了“探索 + 奖励驱动”的好处。在 ALFWorld 这种只有二元奖励的环境里，这个优势尤其突出。
3. **先模仿打底、再试错精炼**：BC 提供可靠起点，ETO 在其上迭代加强。没有 BC 打底时（自我对弈），要先用 RFT 让模型“能跑出成功案例”，再上 ETO。
4. **迭代有甜区、别贪多**：在固定专家集 + 同一训练集上探索，迭代 2 轮左右最好，超过会因数据多样性不足而**过拟合退化**。
5. **粒度选择：整条轨迹对比 > 单步对比**：因为只用最终奖励难以准确判断“单步动作”的好坏（信用分配难题），步级对比反而不稳。
6. **基础语言能力 ≠ 智能体能力**：更强的基座 LLM 不一定造出更强的智能体（Mistral-7B 反不如 Llama-2-7B）。

8.2 对 AI 应用开发的实际启示

- **别只喂“正确答案”给你的 Agent**：如果你在用开源模型搭业务智能体（网页操作、工具调用、流程自动化），**把它线上/测试时跑挂的轨迹收集起来，和成功轨迹配成对做 DPO**，往往比只堆更多“标准示范”更划算、提升更稳。

- **优先考虑离线偏好优化 (DPO) 而非在线 RL**：在线 RL (PPO) 调参痛苦、容易崩，尤其当你的环境只能给“成/败”这种粗奖励时。**DPO + 成败对比**是更工程友好、更稳的替代方案。
- **“失败数据”是被低估的资产**：日志里那些失败 case 不是垃圾——**它们是免费的、最贴近你真实分布的训练信号**。建立一条“收集失败 → 配对成功 → 周期性 DPO 微调”的流水线，可以让智能体随使用越用越好。
- **冷启动策略**：手上没有专家轨迹时，先用拒绝采样 (RFT，只保留自己跑成功的轨迹再训) 做冷启动，再叠加失败对比学习。
- **别盲目迭代**：周期性再训练有“甜区”，盯住验证集，过拟合退化时及时停。

8.3 作者坦承的局限

1. **信用分配过于简化**：ETO 假设智能体“从一开始就走错了”，于是整条失败轨迹对整条成功轨迹比。但现实中智能体往往是**中途某一步**才走错；理想做法是定位到那一坏步、只为后续补专家轨迹。可惜**多数环境不提供这种过程级信息**，难以做步级/过程级奖励建模。潜在方案：用 GPT-4 来识别坏动作、构造细粒度对比数据。
2. **偏专用、未做通用智能体**：本工作主要针对**单一具体任务**训练专用智能体，对“强通用智能体”探索不多。未来会研究 ETO 训出的策略的**可迁移性**、以及多任务训练。

附录要点 (精简)

附录 A (数据集细节)：WebShop 在选“buy”后按属性/价格匹配给奖励；ScienceWorld 含 10 类任务、按子目标给奖励，测试集是“烧开水 → 熔化铅”这类未见变体；ALFWorld 是 ALFRED 的文字版，分 seen/unseen。CoT 标注：WebShop/ALFWorld 有少量人工轨迹，并用 GPT-4 当教师智能体（取奖励 >0.7 的轨迹）；ScienceWorld 用启发式搜索生成黄金轨迹，再用 GPT-4 补 CoT。

附录 B (成功率, 表 7)：除平均奖励外另报成功率，ETO 同样全面领先（如 WebShop 37.5、ScienceWorld-Seen 80.3）。

附录 C (步级对比细节)：对前 $t-1$ 步用 teacher forcing 喂专家动作，让智能体只预测第 t 步，再按最终奖励把第 t 步的 $(a_t, \hat{a}_t)$ 构造成“好—坏动作对” $a_w \succ a_l \mid u, e^{(t-1)}$ ，套进 DPO (公式 13)。

附录 D (案例研究, 图 5/6)：WebShop 里 SFT 智能体漏选了“3pc”颜色选项导致只拿 0.5 分，ETO 学会正确选择属性拿满分；ScienceWorld 里 SFT 智能体执着于错误动作（盯着“手指画”这种非生物），且动作和自己的 CoT 矛盾，而 ETO 正确找到“青蛙卵 → 普通蟾蜍”这一动物完成任务。

附录 E (评测提示词, 图 7/8/9)：给出 WebShop、ScienceWorld、ALFWorld 三个环境的指令提示词模板（动作格式、可用动作列表、Thought/Action 输出格式等）。